



Understanding Large Language Models

A practical primer on how LLMs work — from tokens and embeddings to transformers and temperature. Designed for tech-savvy learners and professionals who want to understand the machinery behind AI-powered language tools.

AI FUNDAMENTALS

LLM GUIDE

What is an LLM?

A **Large Language Model (LLM)** is an AI model trained on massive amounts of text data to understand language, generate human-like responses, and predict the next word or token. It can answer questions, write code, summarise content, and translate languages — all from a single architecture.

Why "Large"?

- Trained on huge datasets — books, websites, code, and articles
- Contains billions to trillions of learnable parameters
- Requires significant computational power (GPUs/TPUs)

Popular LLMs

ChatGPT

Gemini

Claude

Llama

DeepSeek

Real-World Use Cases

Use Case	Example
Chatbots	ChatGPT conversational AI
Coding Assistant	GitHub Copilot
Translation	Google Translate
Content Writing	Blog and article generation
Customer Support	AI-powered support agents

LLM Anatomy: Core Components

LLMs work through four interconnected stages. Each stage transforms raw human input into a structured, intelligent response.

1

Input Processing

Receives and prepares user input — cleaning text, splitting it into tokens, and converting it to a machine-readable format. Example: "I love AI" becomes ["I", "love", "AI"].

2

Model Architecture

The internal structure — transformers, neural networks, and attention mechanisms. Decides how the model learns, understands context, and generates accurate responses.

3

Training Process

The model learns patterns from data — grammar, sentence structure, context, and meaning relationships. Training on coding data enables code generation; medical data enables healthcare Q&A.

4

Output Generation

Predicts the most probable next word. Example: "The cat sat on the" → "mat" (90% probability).

Tokens: The Building Blocks of LLMs

Tokens are the **smallest units processed by an LLM**. A token can be a full word, part of a word, a space, or even punctuation. LLMs cannot process raw sentences — they convert text → tokens → numbers → predictions.

Example: "ChatGPT is amazing!" may split into ["Chat", "GPT", "is", "amazing", "!"]

One Word = Multiple Tokens

"unbelievable" → ["un", "believ", "able"]

Spaces & Punctuation Count

"Hello world!" ≈ 3 tokens

Why Tokens Matter

Tokens directly affect **cost, response length, and performance**.

- More tokens = higher API cost
- More tokens = longer processing time
- Token limits constrain output length

Recommended Token Sizes

Task	Tokens
JSON response	50–100
Summary	150–250
Explanation	300–500
Code generation	500–1000

Embeddings: Converting Words to Vectors

LLMs understand **numbers, not words**. Embeddings convert text into numerical vectors — placing semantically similar words close together in multi-dimensional space. This enables models to understand meaning, compare similarity, and retrieve relevant information.



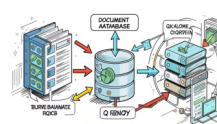
Semantic Search

Find documents or text with similar meaning, not just matching keywords.



Recommendation Systems

Suggest related items by comparing embedding similarity between user preferences and content.



RAG Systems

Retrieve relevant documents. Example: "What is Selenium?" → finds Selenium docs, testing articles, and tutorials.



Chatbots

Understand conversational context and respond meaningfully across multi-turn interactions.

Transformers: The Backbone of Modern LLMs

Transformers are **deep learning architectures** designed to understand relationships between words. They are the foundation of ChatGPT, Gemini, Claude, and Llama.

Older models (like RNNs) processed words *sequentially* — one at a time. Transformers changed everything by processing all words **in parallel**.

→ Parallel Processing

All tokens processed simultaneously

→ Long Context Understanding

Remembers relationships across long text

→ Faster Training

More efficient than sequential models

→ Better Accuracy

Superior results on language tasks

Transformer Architecture Flow

01

Tokenise Input

02

Embed Tokens

03

Self-Attention Layer

04

Feed Forward Network

05

Output Probabilities

Self-Attention & Feed Forward Networks

Self-Attention Mechanism

Self-attention helps the model decide **"Which words are important?"** The same word can have different meanings depending on context.



Example:

Sentence 1: *"I went to the bank to deposit money."*

Sentence 2: *"I sat near the bank of the river."*

Self-attention checks nearby words like deposit money → model understands bank = financial institution.

Feed Forward Neural Network (FFN)

After attention processing, the FFN **further refines understanding** and strengthens important patterns.

Attention

Identifies word relationships

FFN

Extracts meaning and improves prediction quality

Together, attention and FFN layers are stacked multiple times in a transformer — each layer deepening the model's understanding.

Output Generation Strategies

Once the model has processed your input, it must decide **how to generate the response**. Different decoding strategies trade off between speed, accuracy, and creativity.



Greedy Search

Always chooses the **highest probability word** at every step. Fast and deterministic — ideal for factual responses and structured output.

*Example: "The cat sat on the **mat**"*



Beam Search

Evaluates **multiple sentence possibilities** simultaneously before selecting the best sequence. Improves coherence and overall response quality.

Example: Evaluates "mat", "chair", "roof" before committing.



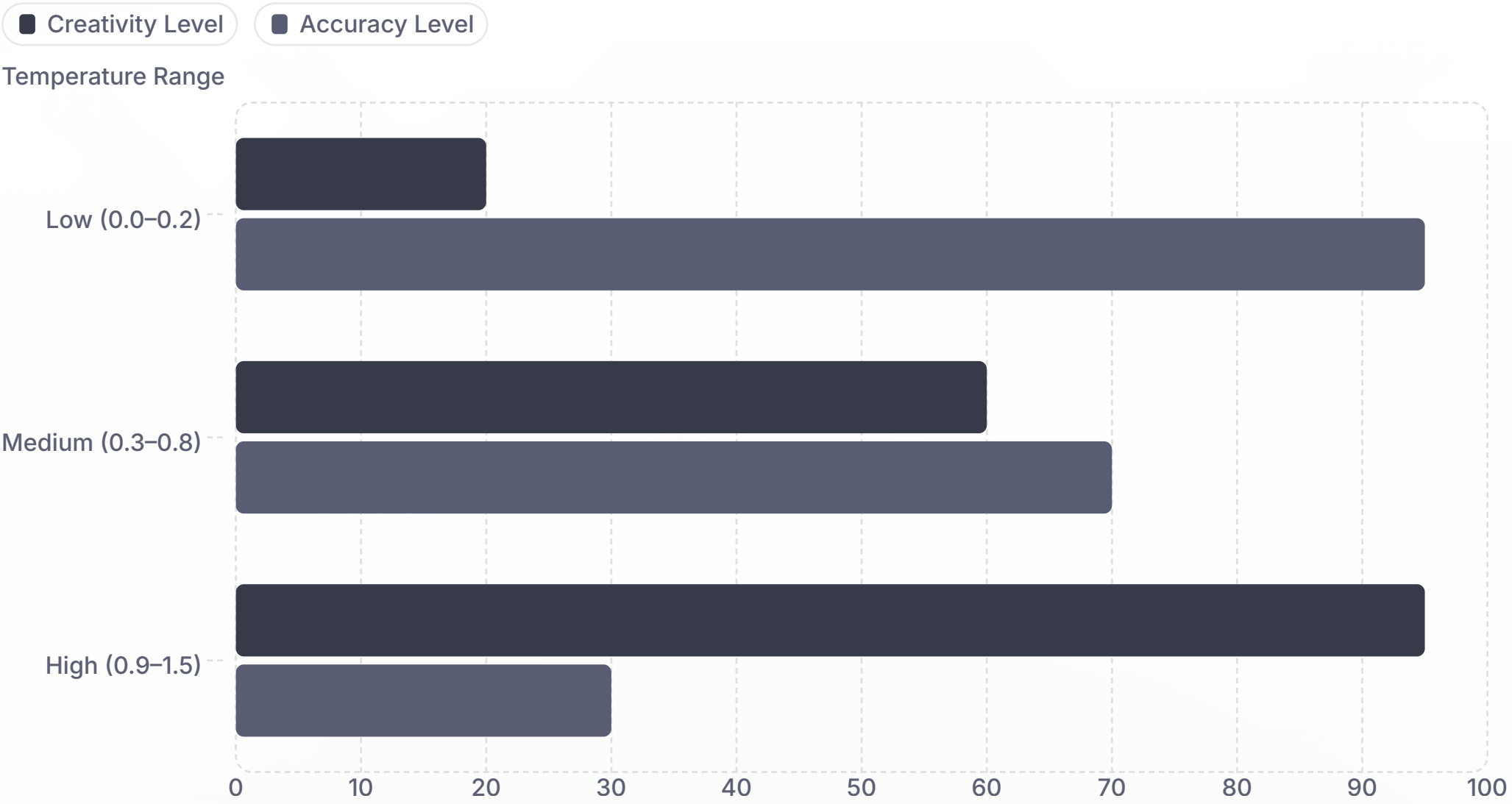
Sampling

Randomly chooses words based on probability distributions. Adds creativity and diversity — used in story generation, brainstorming, and marketing content.

Temperature: Controlling Creativity

Temperature is a hyperparameter that controls the **randomness and creativity** of LLM outputs. Adjusting it changes how predictable or surprising the model's responses will be.

 Low (0.0 – 0.2) Use for: JSON APIs, code generation, factual Q&A Ensures accurate, consistent, and stable output. Minimal randomness.	 Medium (0.3 – 0.8) Use for: Chatbots, tutorials, customer support Balances accuracy with natural, conversational tone.	 High (0.9 – 1.5) Use for: Story writing, brainstorming, marketing Encourages creativity and diverse, unexpected outputs.
---	--	--



Key Takeaways

Here is a concise summary of the core concepts that power every Large Language Model you interact with.



Tokens

Split text into the smallest processable units. Directly impact cost and performance.



Embeddings

Convert text into numerical vectors so models can understand meaning and similarity.



Transformers

The backbone architecture — processes all tokens in parallel for speed and context awareness.



Self-Attention

Identifies which words matter most in a given context, resolving ambiguity.



FFN

Refines understanding after attention, strengthening patterns for better predictions.



Temperature

Controls the creativity vs. accuracy trade-off in generated outputs.



Next Steps: Experiment with LLM APIs (OpenAI, Anthropic, Google), explore Hugging Face models, and try building a simple RAG pipeline to apply these concepts in practice.